LAB 7 | Fuzz Testing for Input Validation Bugs

Student: S Bhuvanashree **SRN:** PES1UG24AM235

Assigned application: Bike Share Rental | **Reference:** SETAPESU26/L6_L7_53_bike_share_rental

Objective

Use Python Hypothesis property-based fuzz testing to detect invalid-input failures in three functions, fix input validation, and demonstrate both the failures and successful retesting.

Functions tested and fixes implemented

Function	Bug / invalid input	Updated behavior
parse_ride_duration()	Missing values, malformed HH:MM, invalid minute fields	Return 0 for invalid input; validate numeric hours and 00-59 minutes
format_station_code()	Missing or malformed station codes; unsafe string operations	Return empty string for invalid codes; normalize a12 to A-12
parse_ride_log()	Missing CSV fields or nonnumeric minutes; unsafe indexing/conversion	Return ("", 0, False) when invalid; validate three fields

Environment and testing procedure

Run on macOS using Python 3.13.15, pytest 9.2.0, and Hypothesis 6.170.15. Execute the original and fixed program with inputs **01:15**, **a12**, and **electric,45,yes**. Run the same three Hypothesis tests against both program versions.

Command 1: `python bike_share_rental_parser.py`
Command 2: `python -m pytest test_bike_share_rental_parser.py -v`

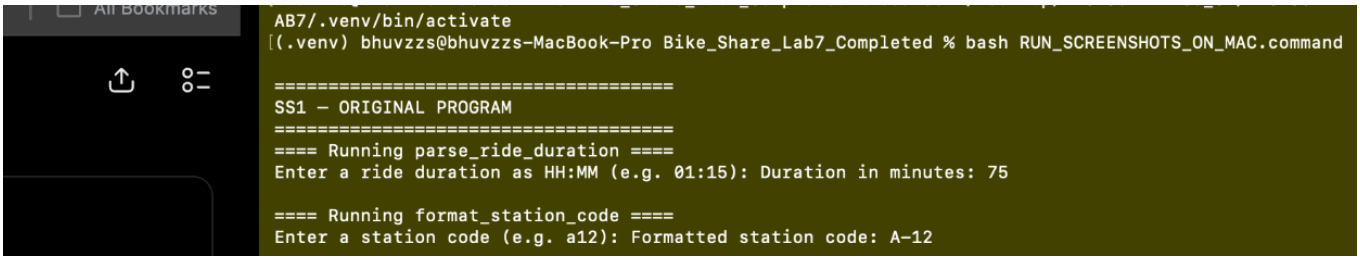
Observed result

Original code: 3 tests failed (0.84 s). **Fixed code:** 3 tests passed (0.92 s).

The screenshots on the following pages are cropped from actual Mac Terminal outputs supplied for this lab; they are not simulated test results.

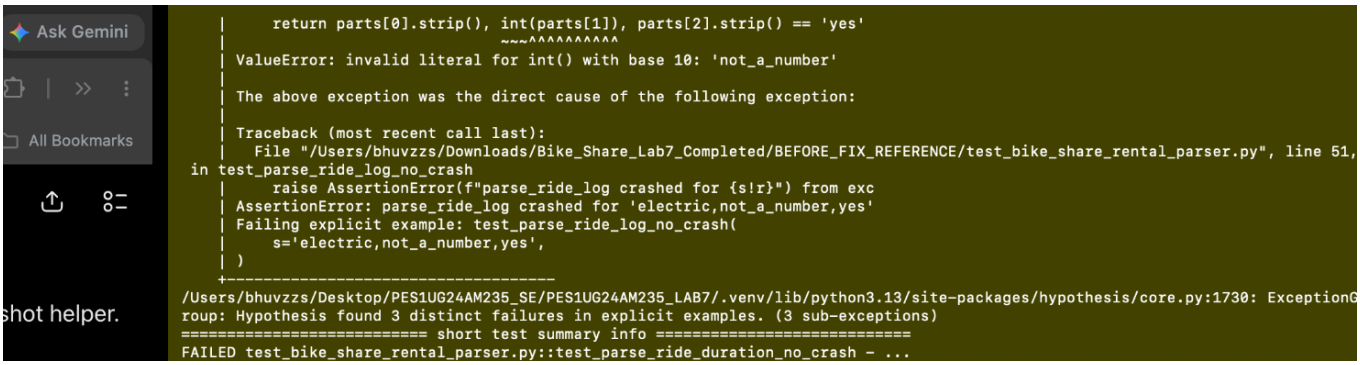
SS1 | Original buggy application

Running the supplied original parser on valid sample input. Result: 75 minutes, A-12, and (electric, 45, True).



SS2 | Failing Hypothesis tests

The unchanged buggy parser fails all three property-based tests; the Terminal summary confirms 3 failed in 0.84 s.



SS3 | Fixed application

Running the corrected parser on the same valid sample input. The expected outputs are retained after input-validation changes.

```
====================================
SS3 — FIXED PROGRAM
====================================
==== Running parse_ride_duration ====
Enter a ride duration as HH:MM (e.g. 01:15): Duration in minutes: 75

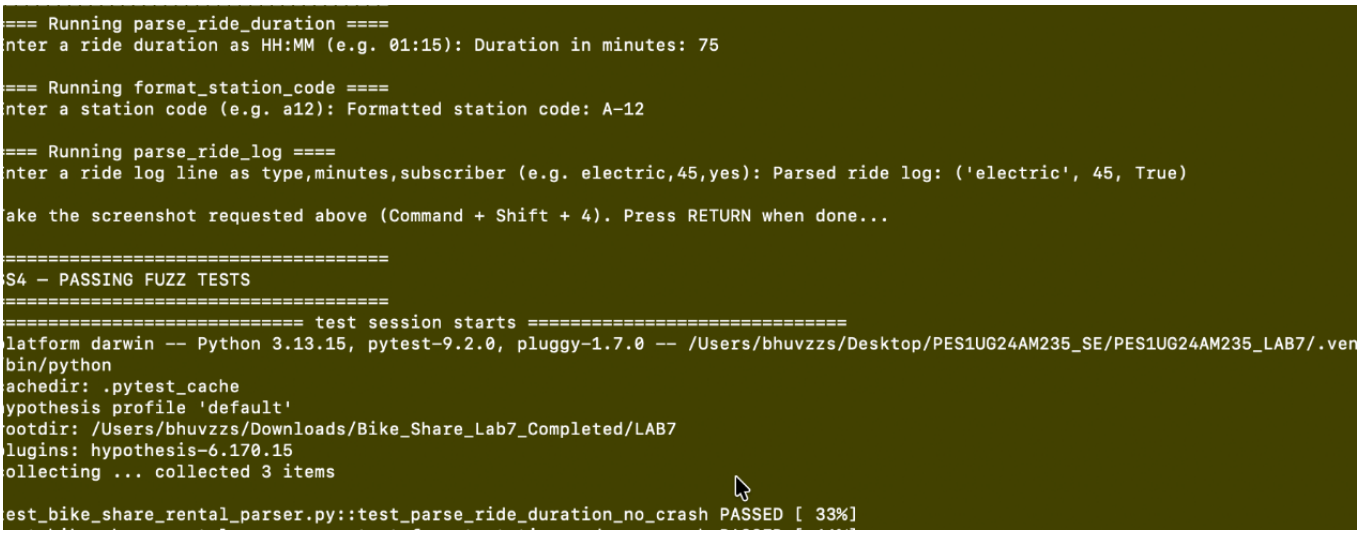
==== Running format_station_code ====
Enter a station code (e.g. a12): Formatted station code: A-12

==== Running parse_ride_log ====
Enter a ride log line as type,minutes,subscriber (e.g. electric,45,yes): Parsed ride log: ('electric', 45, True)

Take the screenshot requested above (Command + Shift + 4). Press RETURN when done...
```

SS4 | Passing Hypothesis tests

The corrected parser completes all three fuzz tests successfully: 3 passed in 0.92 s.



Bug findings, reflections and conclusion

Why did the original implementation fail?

The original parser assumed incoming values were correctly typed and formatted. Hypothesis generated unexpected values such as None, blank strings, and malformed duration, station, and ride-log fields. Direct calls to split(), strip(), indexing, and int() could raise exceptions.

How were the bugs fixed?

The corrected implementation checks the input type and structure before conversion. The duration parser accepts hours and two minute digits (00-59); the station formatter validates and normalizes a letter-plus-digits station code; and the ride-log parser verifies three fields, nonnegative whole minutes, and a yes/no subscriber flag. Each function returns a defined safe fallback on invalid input.

Why use Hypothesis fuzz testing?

Ordinary tests check known inputs. Hypothesis generates many unusual inputs, shrinking failures into small counterexamples that reveal missed assumptions. Passing these properties improves confidence in robustness but does not establish that every conceivable input has been tested.

Test results summary

Phase	Test count	Observed outcome
Before fix	3	3 failed in 0.84 s
After fix	3	3 passed in 0.92 s

Conclusion

Fuzz testing identified crashes caused by invalid input across all three bike-rental parsing functions. After adding defensive validation, the same three Hypothesis tests passed while normal program behavior remained correct.

Submission files

1. **bike_share_rental_parser.py** - fixed module
2. **test_bike_share_rental_parser.py** - completed Hypothesis tests
3. **PES1UG24AM235_Bhuvanashree_LAB7.pdf** - report with SS1-SS4

Note: The uploaded four-page lab handout does not specify the wording of the three reflection questions mentioned in the repository README. The three reflection sections above address the lab findings, fixes, and purpose of fuzz testing.